Aula 01 -> 03/10/26

2> Flexbox

1> Vem para resolver a responsividade.

CSS 3 -> Flexbox, Grid Layout.

2> Surgimento de novas tecnologias.

> Containers.

Flexbox permite que os caixas podem ocupar espaços diferentes forma a depender do conteúdo será utilizado.

* Ideal para conteúdos que vão flexível. -> Portabilidade.

1> Deve haver container e itens.

2> Pai. 1> filho.

Flexbox não define o tamanho fixo.

1> display: flex; -> container.

1> Define a caixa flexível no container;

1> flex: auto; -> itens.

1> Define a caixa auto ajustável do item de acordo com a pai.

• Como o conteúdo será organizado é definido no CSS.

Aula 02 -> 03/10/26

2> Direção e eixos Flexbox.

flex-direction -> Container. > Padrão.

1> 4 tipos: flex-direction: row; -> →

1> Define como o conteúdo será posicionado a sua direção.

row-reverse; ←

column; ↓

column-reverse; ↑

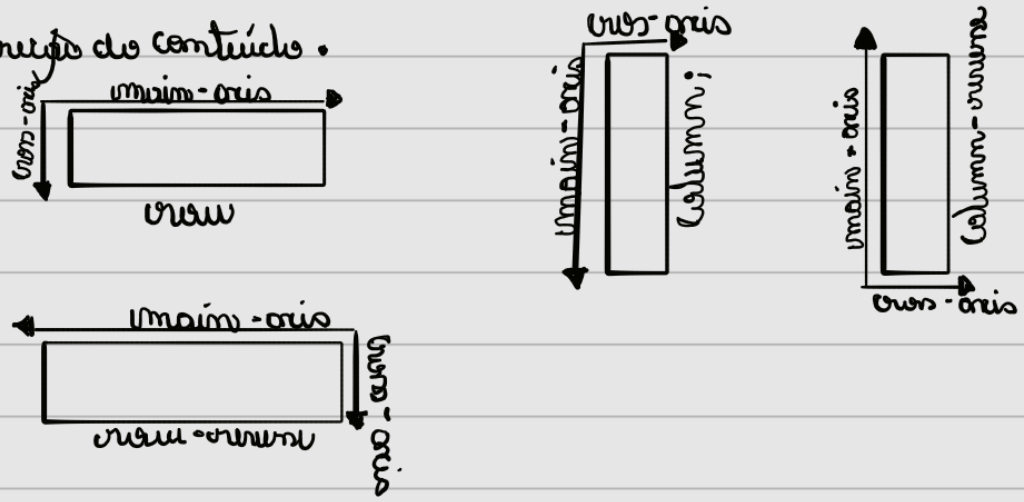
1> Definido as direções podemos os eixos:

2> (axis)

main-axis: Eixo que corta o meio do container na horizontal.

cross-axis: Eixo que corta o meio do item na vertical

main segue a direção do conteúdo.



main-start: Local onde o conteúdo será utilizado.

main-end: Local onde termina o eixo da direção.

↳ Mesma coisa para cross.

- Procura de organizar os eixos e direções dos itens para utilização.

Aula 03 -> 03/10/26

↳ Empacotamento e flex.

- O que é empacotamento?

flex-wrap: Configura o comportamento da caixa pai.

↳ Por padrão: nowrap;

↳ Significa que não quebra os conteúdos.

↳ Encolhimento depende do conteúdo.

flex-wrap: Quando a caixa pai diminui aconteces a quebra de linhas na direção do cross-axis e main-axis.

↳ flex-wrap: wrap;

flex-wrap: wrap-reverse;

↳ O item vai para o inverso do eixo cross-axis.

↳ Continua na direção do main.

↳ flex-flow: Direção quebra de linha;

↳ Shorthand de flex-direction e flex-wrap;

Aula 04 -> 03/10/26

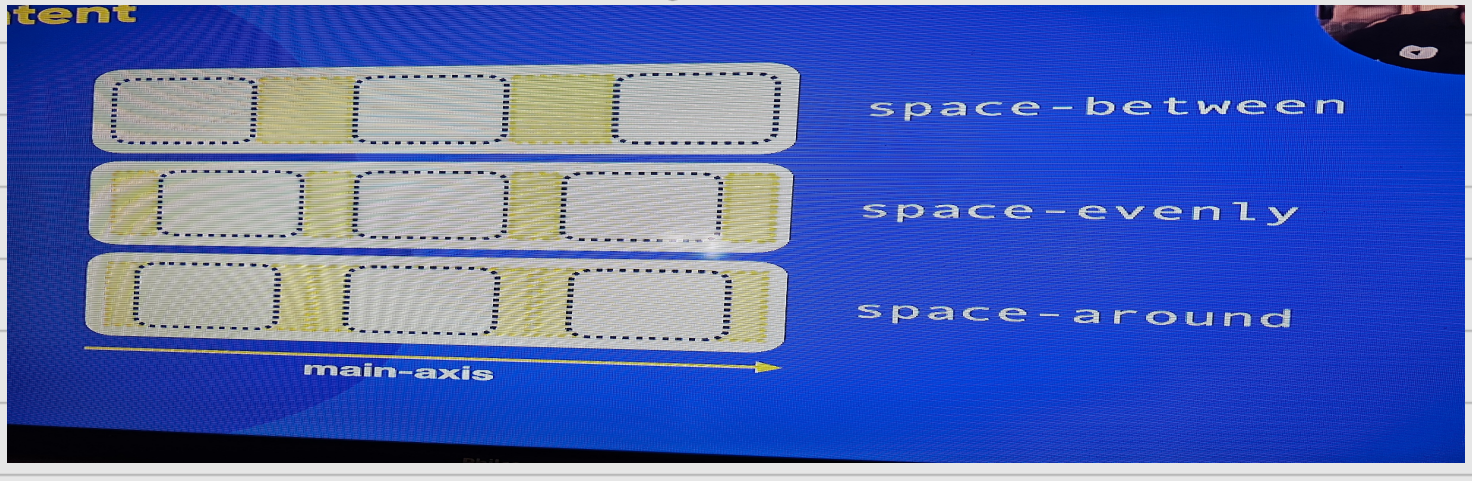
↳ Alinhamento nos eixos Flexbox.

- justify-content: Espaçamento nos eixos.

↳ Valor inserido depende do main-start e main-end.

- Valor padrão.

- flex-start: o espaço em branco sempre virá ficar no final do eixo principal.
  - Itens colados ao main-start.
- flex-end: o espaço em branco sempre no início do eixo principal.
  - Itens colado ao main-end.
- flex-center: virão ficar no meio e os espaços em branco virão ficar as margens.
  - Itens totalmente centralizado.
- space-between: 1º item colado no main-start, o último no main-end, e todos os outros no meio igualmente espaçados entre os dois.
- space-evenly: todos os itens darem exatamente espaçamento iguais.
  - Antes e depois de cada um.
- space-around: os itens ficam calculados ao centro do espaço útil do container pai.
  - Ficam igualmente espaçados.



- align-items: Espaçamento no eixo vertical.
  - Valores são inseridos no eixo cross-axis.
    - Por padrão: stretch: Estica os elementos na direção do cross-axis.
      - flex-start: Espaço em branco no final do cross-axis.
        flex-end: " " " " início do cross-axis.
        center: " " " " início e fim do cross-".
    - cross-start, cross-end.

Aula 03 -> 03/10/16

2. centralização absoluta Flexbox.

2. independente do tamanho do container, o item sempre fica no meio.

justify-content: center;

align-items: center;

2. centralização absoluta.

• Recomenda-se não usar Grady como display-flex

Aula 06 -> 03/10/16

2. Alinhamento de elementos empacotados no Flexbox.

• align-content: Alinha os elementos no eixo transversal, quando somente empacotado.

2. Não é a mesma coisa que justify e align-items.

2. Valor padrão stretch

2. Divide a altura pela a quantidade de linhas ou coluna para dimensionar o tamanho do item.

2. align-content: start;

2. Gruda todos os elementos no eixo-start

2. align-content: end;

2. Gruda todos os elementos no eixo-end.

2. align-content: center;

2. Reserva espaço no eixo-start e eixo-end iguais.

2. align-content: space-between;

2. Primeiro gruda eixo-start, último gruda eixo-end, restante no meio.

space-evenly;

2. Entre os elementos e o eixo-start e end, os espaços ficam entre elementos iguais.

space-around;

2. Divide a quantidade em caixas a quantidade de linhas/colunas e centraliza os itens ao meio.

2. wrap ou wrap-reverse.

Aula 07 -> 06/10/26

↳ Anatomia dos itens.

order: Define a lista dos itens que serão mostrados um ordem, de acordo com o main axis.

↳ Por padrão, todos vão 0.

↳ Vai de -∞ até ∞.

↳ Tem maior prioridade.

align-self: Se aplica a cada um dos itens.

↳ Funciona nos dois eixos - axis.

↳ Valor: auto -> herda o alinhamento do Pai. (container).

start -> colado no start-axis

end -> colado no end-axis

center -> centralizado entre os dois.

stretch -> Grudado no start-axis e end-axis.

align-items: container, Pai. } ↳ mesma declaração.

align-self: item, filho. }

Aula 08 -> 06/10/26

↳ Propriedade flex-basis.

↳ itens do container.

↳ Define o tamanho máximo do item no container.

↳ Valor padrão: auto;

↳ Tamanho do eixo largura ou dado através do tamanho dos conteúdos dentro dele.

↳ Podemos definir com valores não se adapta ao conteúdo.

• Substitui: width, hight. ↳ flex-basis: ... px;

↳ Enquanto o conteúdo for maior permite, porém quando for menor, se adapta.

↳ Tenta não permitir quebra de linhas.

Aula 10 -> 06/10/16

↳ Controle do tamanho dos itens.

↳ flex-grow , flex-shrink ;

↳ O quanto ele pode crescer. ↳ O quanto ele pode diminuir.

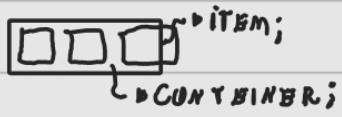
flex-grow : 0; -> Não pode crescer, ≠ 0, pode crescer.

flex-shrink : 1;

↳ Sim, pode encolher; se for 0, não pode.

• Elementos altamente fluídos, crescem e diminuem a vontade.

↳ Quando o item tem shrink : 0, e o container diminui, não diminui junto, e não quebra linha.



• Elementos parcialmente fluídos.

↳ grow não necessariamente funciona com flex-flow: nowrap, a dinâmica é diferente quando há quebra de linha.

• Propriedade não é binária;

↳ Podemos determinar uma propriedade diferente para cada item, cada um com comportamento diferente. # Cria uma ordem de prioridade, onde o número aumenta de acordo com o anterior, de forma proporcional.

↳ 1 2 3. ↳ Segundo cresce o dobro do anterior.

↳ Ordem importa para a proporção.

↳ Comportamento igual para grow e shrink.

• São propriedades que permitem a fluidez dos itens.

Aula 11 -> 18/06/2016

↳ Shorthand : ítens, grow, shrink.

↳ flex: grow shrink ítens ;

↳ Recomenda-se uma ordem pela W3C.

• Evitar declaração dos grow, shrink e ítens.

-> flex: 0 1 auto; -> Valores padrão dos itens ou flex: initial ;

flex: 00 auto; -> Valores não flexível ou flex: none;

flex: 1 1 auto; -> Valores fluídos mais flexível ou flex: auto;

Aula 12 -> 18/10/2016

2. Menu responsivo (Parte II). -> Arquivo/dd

1. Flex - Base para uso de menu responsivo com HTML e CSS básico.

Aula 13 -> 18/10/2016

2. Menu responsivo (Parte II). -> Arquivo/dd

1. Estilização e declaração da estruturas hierarquias dos caixas. Pai e filhos.

Aula 14 -> 18/10/2016

2. Menu responsivo (Parte III). -> Arquivo/dd

Utilização de media query para modulo first.

-> Definição dos tamanhos das telas para utilização.

Aula 15 -> 18/10/2016

2. Conteúdo com flex Base (Parte I) -> Arquivo/dd

1. 100%. apenas com flex-base.

- Formulação da estruturas com HTML

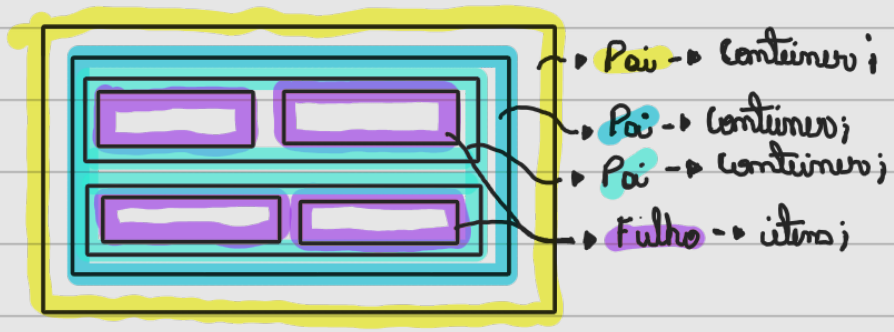
Aula 16 -> 19/10/2016

1. Conteúdo com flex Base (Parte II) -> Arquivo/dd

1. Definição das ordem dos itens com CSS para ajustar na ordem desejada.

1. flex-flow: ... ... ;

1. Define a ajuste do texto.

1. 

1. Estrutura de hierarquias para flex-base.

Aula 17 -> 19/10/2016

1. O que é Grid layout? ≠ Flex Base.

1. Sistema para criação de layout baseado em uma grade lógica -

unercional e otimizado para design de interfaces de usuário. Elementos - filhos do container que define a grade pode ser posicionados livremente nos espaços da estrutura, mesmo quando de elementos flutuantes ou fixos.

↳ Mesmo que defina ainda pode se reorganizar dentro dos medidas.

- Recomenda-se para melhorar a interface do usuário.

↳ Dividimos o site com linhas e colunas e definimos os intervalos que serão mostrado o grid. * Calculo depende da quantidade de L e C.

-> Criar interface e posicionar os elementos do site de maneira organizada.

↳ Função do grid layout.

Aula 18 -> 19/10/2026

1 ▸ Anatomia de um grid layout.

↳ grid row ->

grid line ->

-> Todo grid é iniciado em container.

-> Todo grid possui um item.

-> Grid lines são as linhas que cruzam e demarcam os espaços para formação do grid.

↳ São numerados. 1 -> ∞. E -> D.

↳ São horizontais e verticais.

↳ Numerados de 1 ↓ ∞. Top -> Bottom.

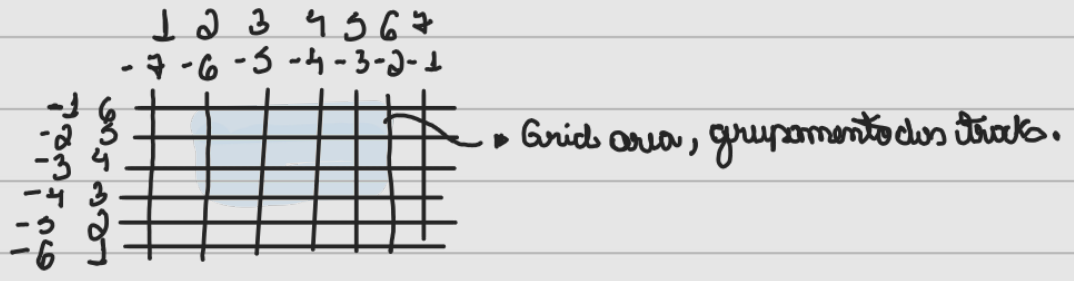
-> Grid track são um conjunto de itens em linha horizontal.

↳ column track. ↳ row track.

↳ Como se cruzam o nível chama-se item de grid

- Numeração pode ser negativa do grid line.

↳ Usado para valermos o caminho de ponta a de trás para frente.



▸ Grids area, grupamento dos tracks.

↳ Espaços entre os grid itens é possível aumentar usando grid gaps.
↳ Não interfere no tamanho do grid item.

Aula 19 → 20/10/2016 → exercícios / ex 02.
↳ Propriedade do container para Grids.
↳ display;
↳ grid-template-columns;
↳ grid-template-rows;
↳ gaps;

↳ Definimos os grid container como display: grid;
↳ não altera nada.
↳ grid-template-columns, definimos a qtd e o tamanho ou fr (fração/proporção) das colunas.
↳ grid-template-rows, mesma definição e uso do anterior mas agora em linhas.
↳ gap, usado para os espaços entre os grid-item.

Aula 20 → 20/10/2016
↳ CSS grid layout (Prática I)
↳ Implementação do container.
ATALHO ! tag. "nome da classe" → cria a classe diretamente.
↳ tag. nome da classe * número → cria a classe e quantas vezes quiser.
• Ele muda as configurações do container.
↳ display: grid;
→ Importância quando usar grid para verificar grid view.
↳ Tecnologia de auxílio.

Aula 21 → 20/10/2016
↳ Propriedades de container para Grid.
↳ align-items; ↕
↳ justify-items; ↔ ↳ align propriedade vertical;

↳ align-content; ↕ ↳ Justify propriedade horizontal;
↳ justify-content; ↔

↳ conjunto inteiro;

↳ align-items ≠ align-content. = Valor padrão = Stretch;
↳ alinhamento do item de acordo com a propriedade.
↳ justify-items: Tem a mesma função na vertical;

↳ align e justify content tem as mesmas propriedades de flexbox.
↳ Vertical; ↳ Horizontal;

Aula 22 -> 20/10/2026

↳ CSS Grid Layout na prática. (Parte II)

content -> Todos os itens agrupados num container.
items -> Itens dentro da div.

↳ Prática para brincar com os espaços do grid.

Aula 23 -> 20/10/2026

↳ Propriedade do container para Grid -> Grid explícito quando os itens preenchem todo o grid.

- -> grid-auto-row;
- -> grid-auto-column;
- -> place-items;
- -> place-content; } Propriedade simplificadoras;
- -> grid-template;

↳ grid-auto-row: Vem para acrescentar mais linhas que não foram definidas no template-rows;

↳ grid-auto-column: Vem para adicionar mais uma coluna que não foram definidas no template-column;

- place-items: Shorthand para declarações justify e align-items.

Ordem de declarações: align-items justify-items;

- place-content: Shorthand para declarações justify e align-content.

Ordem de declaração: align-content justify-content;

• grid-template: Shorthand para declaração de row e column-template.

Ordem de declarações: template-rows / template-columns;

Aula 24 -> 20/10/2026

2> Propriedade de container para Grid

1> fr -> Fração; -> Espaço livre no container.

1> repeat(" ", 1); -> Parâmetros = qtd, tamanho.

• Podemos inputar vários medidas diferentes.

2> Usada para declarações de template definindo a quantidade e o tamanho.

1> repeat(3, auto);

1> Significa: auto auto auto;

• Frações são muito mais práticas, que outras medidas para definição.

1> Considera o tamanho do container e realiza cálculos para adaptação.

1> Quanto maior o fr a proporção vai maior dentro do container.

Aula 25 -> 20/10/2026

2> CSS Grid Layout na prática (Parte III).

1> grid-auto-flow: Determina onde iremos adicionar os itens para do grid area.

2> column ou row;

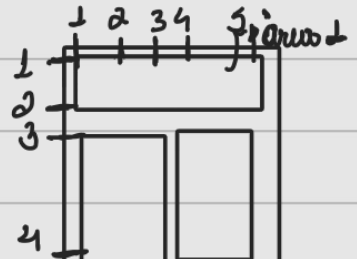
Aula 26 -> 20/10/2026

2> Propriedade de itens para Grid.

-> grid-row-start; } Atributos para definir o tamanho da track, na horizontal.
-> grid-row-end; } -> Espaço preenchido com os grid-item;
-> grid-column-start; } " " " " " " " , na vertical
-> grid-column-end;

1> Posicionamento dos elementos dentro da área.

-> Definição de uma área: grid-row-start: 1;
grid-row-end: 2;
grid-column-start: 1;



grid-column-end: 6;

↳ Definimos a delimitação com as marcações de columns e rows do grid layout container.

Aula 27 -> 20/10/2016

↳ Praticando itens no Grid Layout (Parte IV).

• Delimitar os espaços onde definimos a área do grid com os itens.

Aula 28 -> 20/10/2016

↳ CSS Grid: Shorthands para itens.

↳ grid-row-start/end: Shorthand.

grid-column-start/end: Shorthand.

↳ span: Definição de expansão dos elementos para chegar num final.

grid-area: Definição dos espaços delimitados com a declaração das linhas e colunas.

↳ row-start/column-start/row-end/column-end;

Aula 29 -> 20/10/2016

↳ Desafio: Criação do layout com CSS.

↳ Resolução do desafio -> √, feito.

Aula 30 -> 20/10/2016

↳ Jogo para treinar FlexBox CSS.

Aula 31 -> Respostas do Flexbox - Froggy

Aula 32 -> Jogo para treinar Grid layout - Grid Gridem

Aula 33 -> Respostas do Gridem Layout.

Aula 34 -> Criação portfólio do projeto final.

↳ Montagem de Grid layout com Flex-Box.

} 21/10/2016

} Codepips para treinar.

- Importante para elaboração de um projeto o uso das duas tecnologias.

Aula 35 -> 21/10/2016

↳ Criação do portfólio e org. dos arq.

Aula 36 -> 21/10/2016

↳ Org. dos arquivos dentro do repo.

↳ Configuração do ambiente dos arquivos.

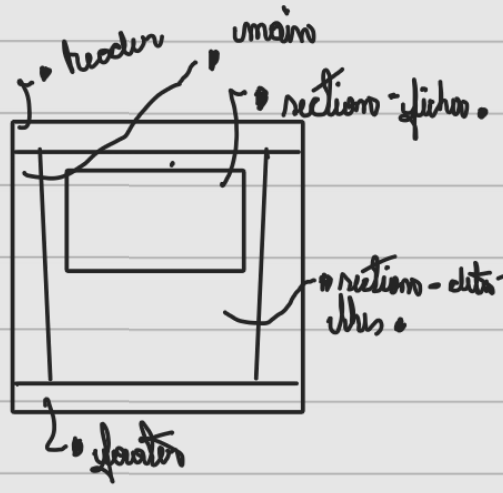
• Arquivos .svg são XML para gerar uma imagem.

}
↳ São bons para gerar imagens pequenas. Possível editar para ajustar a formatação da imagem gerada.

Aula 37 -> 22/10/2026
↳ Menu de portfólio
↳ Criação do menu Flex Box.

Aula 38 -> 22/10/2026
↳ Adição dos itens no menu.

Aula 39 -> 22/10/2026
↳ Construção do Tipo de layout
↳ Apresentação pessoal.
↳ Definição do conteúdo incluído: 2 section ->



header
Main
section
section
Footer
} ESTRUTURA BASE;

-> Estruturas CSS e HTML2 da section da ficha.
aspect-ratio: Proporção do tamanho da imagem incluída;
↳ 1 / 1
↳ □
↳ lados iguais
object-fit: Dimensiona a imagem igual background-size;
↳ cover, contain.

• Técnicas Border Natural do instagram
↳ Border: px tipo de borda tamanho;
outline: " " " " ";

↳ Exemplo.
→ outline
→ border
} ↳ Características de uma caixa: padding, margin, outline, border.

Aula 40 -> 20/10/2026

1. Adicionando vídeo ao portfólio;
    1. header
        nav
            ul
                li
                    a
    main
        section - #hero
            img
            h1
            p
        section -> .datas
            article -> .video
            article -> .formacao
            article -> .skills
            article -> .projetos
    footer

ESTRUTURA GERAL HTML.

1. iframe
    1. caixa container -> onde guardará o vídeo
        1. position: relative;
    1. caixa do item
        1. position: absolute;

main
1. section #datas
    1. article #video
        1. div.video-container
            1. iframe

Aula 41 -> 20/10/2026

1. Barra de progresso das skills.
    1. div#skill-container
        div.item
            div-info
                span.nome
                span.percent } variáveis div-div para não quebrar.
            div.barra
                div-progresso

ESTRUTURA DO ARTICLE SKILL.

1. Width do progresso para cada um é individual, então utilizamos na linha, ou criamos uma classe diferente para todas as skills.

Aula 42 -> 23/10/2026

L> Área de formação

L> article

div#formacao-container

div.curso

p.periodo

h3.instituicao

p.titulacao

L> implementação do CSS.

L> Texto invertido em ordem histórica e column-reverse para mostrar o mais novo para o mais velho.

Aula 43 -> 23/10/2026

L> Área de projetos

L> Definição de ornamentação dos projetos no article.

-> Ornamentação dos projetos em ordem cronológica.

L> Fazer um media query para mudar de acordo com o aparelho.

• Uso de tecnologias Flex-Box.

Aula 44 -> 23/10/2026

L> Header fixo e navegação do site com os itens.

header: position: sticky; -> Fixa a caixa durante a rolagem.

top: 0; -> Posição em relação ao topo.

z-index: 1; -> Camada das caixas. Sobreposição.

L> Por padrão = 0;

• navegação = link -> id do local do site.

L> <a href="#detalhes">

L> Navega até o ponto na pág.

L> JS já usado para calcular altura do header e permitir mostrar a seção que deseja navegar.

L> Uso do JS para mudar as cores padrão definidas no CSS.

↳ Lógica para formatação dos links; para navegação.

Aula 45 -> 23/10/2026

↳ Sticky Header, Grid Título grande, Preparação dos temas.

↳ Definição do # ficha ou ficha na navegação do projeto.

↳ position: sticky;

top: "calculo do total desejado";

↳ Grid Título grande → main → grid-template-column: auto 300px 1fr auto.

↳ Aumento na quantidade de colunas;

↳ Definição das suas posições com grid-column: INICIO/FIM.

↳ definição = #ficha; #detalhes;

↳ Posição responsiva títulos grandes.

Aula 46 -> 23/10/2026

↳ Modo Escuro, Dising responsivo.

↳ Comportamento padrão desvinculado do link.

↳ dom - "link", id = "header-tema"

↳ body.escuro

↳ Verificação com o JS quando a class vai estar escura para adicionar a utilização da class escuro.

• Declaração dos cores CSS.

-> Verificar e adicionar a class escuro no body do HTML.

↳ Quando tiver adicionar class para modificar utilização.